SathiMarket

Authentication & Registration API Guide

VillageSathi — Mobile App Integration (Step by Step)

Generated from villagesathi-ui web codebase

Document date: 16/5/2026

api.villagesathi.in

1. Table of Contents

2. Overview & Base Configuration

All SathiMarket auth APIs use the same axios base URL configured in the web app:

```
Base URL: https://api.villagesathi.in/api
Local dev: https://localhost:7092/api  (see .env VITE_API_URL)

Headers (every request):
  Content-Type: application/json
  Accept: */*

Protected routes (after login):
  Authorization: Bearer <token>
```

Image base URL (shop images, not auth): https://api.villagesathi.in

3. Role IDs

RoleId	Role	Used In
1	Admin	Admin portal — POST /Auth/Login, role check = 1
2	Merchant / Shopkeeper	Merchant login & register — role check = 2
3	Customer	Customer login & register — role check = 3

4. Mobile App — Step-by-Step Build Order

Phase A — Merchant (Shopkeeper) flow

Step 1: On Merchant Register screen load, call GET /Auth/get-roles and GET /Categories/GetAll in parallel.

Step 2: Show Account fields: name, mobileNo, password, roleId dropdown.

Step 3: When user selects roleId = 2, show Shop Information section (shopName, categoryId, shopAddress).

Step 4: On submit, POST /Auth/register with full JSON body (see Section 7–8).

Step 5: On success, navigate to Merchant Login screen.

Step 6: POST /Auth/Login with mobileNo + password. Verify RoleId === 2.

Step 7: Save token + user/shop object. Use shopId for orders, inventory, profile APIs.

Phase B — Customer flow

Step 1: Customer Register — collect identity + address, POST /Customer/Register.

Step 2: Customer Login — POST /Customer/Login. Verify RoleId === 3.

Step 3: Save user profile (userId, name, mobileNo, roleId, city, villageId).

Step 4: Use userId for orders, cart checkout, addresses (ManageAddress API).

5. GET /Auth/get-roles

Purpose: Populate Account Type dropdown on Merchant Register screen.

Request

```
GET /Auth/get-roles
No body. No query parameters.
```

Response (example)

```
[
  { "id": 2, "roleName": "Merchant" },
  { "id": 3, "roleName": "Customer" }
]
```

UI maps: option value = role.id, label = role.roleName. For shopkeeper registration select id = 2.

6. GET /Categories/GetAll

Purpose: Category dropdown inside Shop Information (only when roleId = 2).

Request

```
GET /Categories/GetAll
```

Response (example)

```
{
  "Data": [
    { "categoryID": 1, "categoryName": "Grocery" },
    { "categoryID": 2, "categoryName": "Medical" }
  ]
}
```

Web app also accepts flat array: response.Data || response. Auto-select first category on load.

7. POST /Auth/register

Purpose: Register merchant/shopkeeper (and other roles if API allows). Web uses lowercase path /Auth/register; authApi.js also defines /Auth/Register.

Request — Full merchant payload (roleId = 2)

```
POST /Auth/register
{
  "name": "Ramesh Kumar",
  "mobileNo": "9876543210",
  "password": "SecurePass@123",
  "villageId": 1,
  "roleId": "2",
  "shopName": "Sathi Grocery Store",
  "categoryId": "1",
  "shopAddress": "Main Road, Village XYZ"
}
```

Field reference

Field	Type	Required	Notes
name	string	Yes	Owner full name
mobileNo	string	Yes	10 digits — login ID
password	string	Yes	Plain text over HTTPS
villageId	number	Yes	Web default: 1
roleId	string|number	Yes	2 = Merchant
shopName	string	If roleId=2	Shop display name
categoryId	string|number	If roleId=2	From Categories API
shopAddress	string	If roleId=2	Full physical address

8. Conditional Shop Fields (roleId = 2)

Frontend logic: const isBusiness = Number(formData.roleId) === 2;

When isBusiness is true, render Shop Information block with 3 extra required fields. When false, hide shop UI but formData may still contain empty shop fields on submit.

UI fields when shopkeeper selected

UI Label	JSON key	Control
Shop Name	shopName	Text input
Category	categoryId	Select from Categories/GetAll
Physical Address	shopAddress	Textarea

Success response

```
{ "success": true, "message": "Registration successful" }
// OR
{ "Success": true, "Message": "..." }
```

Error response

```
{ "message": "Mobile already registered" }
```

On success: show toast ' navigate to /merchant-login (mobile: MerchantLogin screen).

9. POST /Auth/Login (Merchant)

Purpose: Unified login used by Merchant and Admin. Merchant app must reject users where RoleId !== 2.

Request

```
POST /Auth/Login
{
  "mobileNo": "9876543210",
  "password": "yourPassword"
}
```

Success response (example)

```
{
  "Success": 1,
  "Message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "Data": {
    "RoleId": 2,
    "ShopId": 15,
    "name": "Ramesh Kumar",
    "mobileNo": "9876543210",
    "shopName": "Sathi Grocery Store"
  }
}
```

Accept both casings: Success/success, Data/data, RoleId/roleId, ShopId/shopId.

Mobile: if Number(data.RoleId || data.roleId) !== 2 ⁉ show "Merchant account only".

Store: token + user object. Dashboard uses shopId for /Orders/GetByShop, /ShopItems/GetByShop, /Shops/GetById.

10. POST /Customer/Register

Request

```
POST /Customer/Register
{
  "name": "Aman Singh",
  "mobileNo": "9876543211",
  "password": "secret123",
  "villageId": 0,
  "fullAddress": "House 12, Ward 3",
  "landmark": "Near Temple",
  "city": "Barabanki",
  "state": "Uttar Pradesh",
  "pincode": "225001"
}
```

Field	Validation (web UI)
name	Required
mobileNo	10 digits

Field	Validation (web UI)
password	Min 6 characters
fullAddress, city	Required
pincode	6 digits
landmark	Optional
state	Default: Uttar Pradesh

Success

```
{ "Success": 1, "Message": "Account created successfully" }
```

11. POST /Customer/Login

Request

```
POST /Customer/Login
{ "mobileNo": "9876543211", "password": "secret123" }
```

Success — flat response (web reads top-level fields)

```
{
  "Success": 1,
  "UserId": 101,
  "Name": "Aman Singh",
  "MobileNo": "9876543211",
  "RoleId": 3,
  "RoleName": "Customer",
  "City": "Lucknow",
  "VillageId": 1
}
```

Mobile: if RoleId !== 3 ⁉ deny access. Save: userId, name, mobileNo, roleId, roleName

Note: Web stores customer in customerUser localStorage; token may not be in response — confirm with backend for mobile Bearer auth.

12. Error Handling & Response Conventions

Check	Meaning
Success === 1	Customer APIs success flag
success === true	Auth register / some login responses
Success === 1 || success === true	Use OR for login/register
message / Message	Error text — show to user
HTTP 4xx/5xx	Use response.data.message or .title

Always send JSON body as UTF-8. Use HTTPS in production. Validate 10-digit mobile and role before navigating to role-specific home.

13. Token & Storage (Web vs Mobile)

User type	Login API	Storage key (web)	Role check
Merchant	/Auth/Login	user + token	RoleId === 2
Customer	/Customer/Login	customerUser	RoleId === 3
Admin	/Auth/Login	user + token	roleId === 1

Mobile recommendation: SecureStore/Keychain for token; separate keys merchantUser vs customerUser. Attach Bearer token on all protected API calls.

14. Quick Reference — All Auth Endpoints

#	Method	Endpoint	Screen
1	GET	/Auth/get-roles	Merchant Register
2	GET	/Categories/GetAll	Merchant Register (shop)
3	POST	/Auth/register	Merchant Register submit
4	POST	/Auth/Login	Merchant Login
5	POST	/Customer/Register	Customer Register
6	POST	/Customer/Login	Customer Login